Module 3 — Java 8+ (Lambdas, Functional Interfaces, Streams, Optional)

> Every modern Java interview has a “write this with streams” coding round. Service companies ask stream/lambda syntax; product companies ask stream *internals* and lazy evaluation.

Node.js bridge: Java streams ≈ JS array methods (`map`, `filter`, `reduce`) but **lazy** and one-shot. Lambdas ≈ arrow functions. Functional interfaces ≈ callback type signatures.

3.1 Lambdas & Functional Interfaces

1. Why Interviewers Ask This

Foundation for streams and modern Spring (functional endpoints, `Optional`, callbacks).

2. Core Concept

- A **functional interface** = exactly one abstract method (`@FunctionalInterface`). May have `default`/`static` methods.
- A **lambda** is a concise implementation of that single method: `(args) -> body`.
- **Method reference** = shorthand for a lambda that just calls an existing method: `String::toUpperCase`.

3. Internal Working

Lambdas are **not** anonymous inner classes. `javac` emits an `invokedynamic` bytecode; at first execution `LambdaMetafactory` spins up the implementation class (often reusing a singleton for stateless lambdas). Result: lower overhead and no extra `.class` file per lambda. A lambda captures **effectively final** local variables (copied), and `this` refers to the **enclosing** instance (unlike anonymous classes).

4. Memory Diagram

```
Runnable r = () -> log(x);   // x must be effectively final (captured by value)
   javac -> invokedynamic -> LambdaMetafactory -> synthetic impl (cached if stateless)
   'this' inside lambda == enclosing object (NOT a new anonymous instance)
```

5. The 4 core functional interfaces (memorize)

Interface	Signature	Meaning	Example
`Function<T,R>`	`R apply(T)`	transform	`s -> s.length()`
`Predicate<T>`	`boolean test(T)`	condition	`s -> s.isEmpty()`
`Consumer<T>`	`void accept(T)`	side effect	`System.out::println`
`Supplier<T>`	`T get()`	produce	`() -> new ArrayList<>()`

Also: `BiFunction`, `BinaryOperator`, `UnaryOperator`, `BiPredicate`, `BiConsumer`, and primitive variants (`IntFunction`, `ToIntFunction`, `IntPredicate`) to avoid boxing.

6. Most Asked Questions

- What is a functional interface? Name the built-in ones.
- Lambda vs anonymous class? *(this scope, invokedynamic, no separate class)*
- What is “effectively final”? *(captured locals can’t be reassigned)*

- 4 kinds of method references? *(static `C::m`, instance-of-object `obj::m`, instance-of-type `C::m`, constructor `C::new`)*

7. Traps

- Thinking a lambda's `this` = the lambda (it's the enclosing object).
- Trying to mutate a captured local (compile error).
- Overusing lambdas where a named method is clearer.

8. Best Answer

> "A functional interface has one abstract method; a lambda implements it and compiles to `invokedynamic` via `LambdaMetafactory`, not an inner class — so `this` is the enclosing object and stateless lambdas are cached. Captured locals must be effectively final. I lean on `Function`, `Predicate`, `Consumer`, `Supplier` and method references for readability."

9. Coding Example

```
Function<String,Integer> len = String::length;              // method reference
Predicate<String> nonEmpty = s -> !s.isEmpty();
Supplier<List<String>> newList = ArrayList::new;            // constructor ref
Consumer<String> print = System.out::println;
BiFunction<Integer,Integer,Integer> add = Integer::sum;
System.out.println(len.andThen(n -> n * 2).apply("hello")); // 10
```

10. Follow-ups

- Compose predicates with `.and()/.or()/.negate()`.
- Write a generic `retry(Supplier<T>, int)` helper.

11 & 12. Summary + Cheat

FI = 1 abstract method; lambda = its impl via invokedynamic. Function/Predicate/Consumer/Supplier.

3.2 Streams — the flagship Java 8 topic

1. Why Interviewers Ask This

"Rewrite this loop with streams" and "what does this stream print" appear in nearly every coding round.

2. Core Concept

A **Stream** is a pipeline over a data source: **source → intermediate ops → terminal op**. Streams don't store data, don't mutate the source, and are **single-use** (consumed once).

3. Internal Working — laziness & fusion

- **Intermediate** ops (`map`, `filter`, `sorted`, `distinct`, `limit`, `flatMap`, `peek`) are **lazy** — they build a pipeline and do nothing until a terminal op runs.
- **Terminal** ops (`collect`, `forEach`, `reduce`, `count`, `findFirst`, `anyMatch`, `toList`) trigger execution.
- Elements are pushed **one at a time** through the whole chain (**loop fusion**) — not materialized between stages. This enables **short-circuiting** (`findFirst`, `limit`, `anyMatch` stop early) and lazy `map` calls that never run for skipped elements.
- **`parallelStream()`** splits work across the common ForkJoinPool. Use only for large, CPU-bound, stateless, associative work — and never mutate shared state.

4. Memory Diagram

```
source -> filter -> map -> collect
[1,2,3,4] each element flows fully through the chain before the next:
  1 -> filter(even?) drop
  2 -> filter ok -> map(*10)=20 -> collector
  3 -> drop
  4 -> ok -> 40 -> collector      => [20,40]
Nothing runs until collect() (terminal). limit(1) would stop after first match.
```

5. Key operations

- `map` — 1:1 transform. `flatMap` — 1:many, flattens nested streams (`List<List<T>>` → `Stream<T>`).
- `filter` — keep matching. `reduce` — fold to a single value.
- `collect(Collectors...)` — `toList`, `toSet`, `toMap`, `joining`, `groupingBy`, `partitioningBy`, `counting`, `summingInt`, `averagingDouble`.
- `groupingBy` — `Map<K, List<V>>` (SQL GROUP BY). `partitioningBy` — `Map<Boolean, List<V>>` (split by predicate).

6. Most Asked Questions

- Intermediate vs terminal ops? Give examples.
- Are streams lazy? What triggers execution?
- `map` vs `flatMap`?
- Can you reuse a stream? *(No — `IllegalStateException`.)*
- When parallel streams? Risks? *(shared mutable state, small data overhead, ordering.)*
- `Collectors.toMap` duplicate key behavior? *(throws unless you pass a merge function.)*

7. Traps

- Reusing a consumed stream.
- Side effects in `map`/`peek` (should be pure).
- `toMap` without a merge function on duplicate keys → `IllegalStateException`.
- Assuming parallel streams are always faster (usually slower for small/IO work).
- `forEach` on a parallel stream expecting order (use `forEachOrdered`).

8. Best Answer

> “A stream is a lazy pipeline: intermediate ops just describe work and only run when a terminal op pulls elements through, one at a time with loop fusion, which lets short-circuit ops like `findFirst` stop early. `map` is 1:1, `flatMap` flattens nested structures. I use `groupingBy`/`partitioningBy` for aggregation and avoid parallel streams unless the workload is large, CPU-bound, and stateless.”

9. Coding Example

```
record Emp(String name, String dept, int salary) {}
List<Emp> emps = ...;

// group salaries by department
Map<String, List<Emp>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept));

// average salary per department
Map<String, Double> avg = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept, Collectors.averagingInt(Emp::salary)));

// highest paid per department
Map<String, Optional<Emp>> top = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept,
             Collectors.maxBy(Comparator.comparingInt(Emp::salary))));

// names of employees earning > 100k, sorted, comma-joined
String csv = emps.stream()
    .filter(e -> e.salary() > 100_000)
    .map(Emp::name).sorted()
    .collect(Collectors.joining(", "));

// partition into high/low earners
Map<Boolean, List<Emp>> parts = emps.stream()
    .collect(Collectors.partitioningBy(e -> e.salary() > 100_000));

// flatMap: all distinct chars across names
List<Character> chars = emps.stream()
    .flatMap(e -> e.name().chars().mapToObj(c -> (char) c))
    .distinct().toList();
```

10. Follow-up Coding Questions

- Frequency count with `Collectors.groupingBy(x, counting())`.
- Second highest salary using streams.
- Sum of squares of even numbers with `reduce`/`sum`.
- Flatten `List<List<Integer>>` to a sorted distinct list.
- Convert `List<Emp>` to `Map<name, salary>` handling duplicate names.

11 & 12. Summary + Cheat

Lazy pipeline, single-use, terminal triggers. map/flatMap/filter/reduce/collect; groupingBy/partitioningBy for aggregation.

3.3 Optional

Core Concept & Internal Working

A container that may or may not hold a non-null value — designed to make “absence” explicit at the type level and reduce NPEs. Intended primarily as a **return type**, not for fields or parameters.

API you must know

`Optional.of(x)` (x must be non-null), `ofNullable(x)`, `empty()`, `isPresent()/isEmpty()`, `get()` (avoid), `orElse(default)`, `orElseGet(supplier)`, `orElseThrow()`, `map`, `flatMap`, `filter`, `ifPresent(consumer)`, `ifPresentOrElse(...)`.

Best practices / traps

- **Never** call `get()` without checking — defeats the purpose and NPE-equivalent.
- **`orElse` vs `orElseGet`:** `orElse(expensive())` **always** evaluates the argument even when present; `orElseGet(() -> expensive())` is lazy. Use `orElseGet` for costly defaults.

- Don't use `Optional` for fields, method params, or collections (use empty collections instead).
- Don't do `if (opt.isPresent()) opt.get()` — use `map`/`ifPresent`/`orElse`.

```
// Spring Data returns Optional:
User user = repo.findById(id)
    .orElseThrow(() -> new NotFoundException("user " + id));

String city = repo.findById(id)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("UNKNOWN");               // null-safe chain, no NPE
```

Best Answer

> "`Optional` makes absence explicit in the return type so callers must handle it. I chain `map`/`flatMap` for null-safe navigation and use `orElseThrow`/`orElseGet` — never bare `get()`. I avoid it for fields and parameters; that's an anti-pattern."

3.4 Method References (quick reference)

Kind	Syntax	Lambda equivalent
Static	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Instance of a *particular* object	`System.out::println`	`x -> System.out.println(x)`
Instance of an *arbitrary* object of a type	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Module 3 — Top 25 Interview Questions (senior answers)

1. **Functional interface?** One abstract method; `@FunctionalInterface`.
2. **Built-in functional interfaces?** Function, Predicate, Consumer, Supplier (+ Bi/primitive variants).
3. **Lambda vs anonymous class?** invokedynamic, enclosing `this`, no extra class.
4. **Effectively final?** Captured locals can't be reassigned.
5. **Method reference kinds?** static, bound instance, unbound instance, constructor.
6. **What is a Stream?** Lazy single-use pipeline over a source.
7. **Intermediate vs terminal ops?** Lazy builders vs execution triggers.
8. **Is stream lazy? Proof?** Yes — `peek`/`map` don't run until terminal; short-circuits stop early.
9. **map vs flatMap?** 1:1 vs 1:many flatten.
10. **reduce use?** Fold to single value with identity + accumulator (+ combiner for parallel).
11. **collect / Collectors?** toList/toMap/joining/groupingBy/partitioningBy/counting.
12. **groupingBy vs partitioningBy?** Arbitrary key map vs boolean 2-way split.
13. **Can you reuse a stream?** No — IllegalStateException.
14. **Parallel stream risks?** Shared mutable state, overhead, ordering, common pool starvation.
15. **toMap duplicate keys?** Throws unless merge function provided.
16. **Stream vs Collection?** Computation pipeline vs data storage.
17. **Optional purpose?** Explicit absence, fewer NPEs; return type only.
18. **orElse vs orElseGet?** Eager arg vs lazy supplier.
19. **Why not Optional fields/params?** Overhead + not designed for it; use empty collections.
20. **findFirst vs findAny?** Deterministic first vs any (parallel-friendly).

21. **peek use/abuse?** Debugging only; not for side effects driving logic.
22. **mapToInt/IntStream?** Avoid boxing; get `sum/average/summaryStatistics`.
23. **Collectors.joining?** Concatenate with delimiter/prefix/suffix.
24. **How does parallelStream split?** Spliterator + common ForkJoinPool.
25. **Infinite streams?** `Stream.iterate/generate` + `limit` (must short-circuit).

Module 3 — Top Coding Questions

- Group employees by dept; average/max salary per dept.
- Find duplicates / first non-repeating char with streams.
- Second highest number; top-K frequent words.
- Flatten nested lists (`flatMap`); merge maps.
- Word frequency count; sort a `Map` by value.
- Partition numbers into even/odd; sum with `reduce`.

Module 3 — Common Follow-ups

- "Show it with a `for` loop, then streams — which is clearer here?"
- "Would you parallelize this? Why/why not?"
- "How does `Collectors.toMap` behave on collisions and how do you fix it?"

Module 3 — One-Page Cheat Sheet

```
Functional interface = 1 abstract method. Lambda -> invokedynamic (not inner class), enclosing this
Function(apply) Predicate(test) Consumer(accept) Supplier(get)
Method refs: Class::static, obj::method, Class::instanceMethod, Class::new
Stream = lazy, single-use pipeline. Intermediate(map/filter/flatMap/sorted/limit) lazy;
Terminal(collect/reduce/forEach/count/findFirst/anyMatch) triggers.
map=1:1, flatMap=flatten. Collectors: toList/toMap(merge!)/joining/groupingBy/partitioningBy/counting
Optional: return type only; map/flatMap/orElseThrow; orElse(eager) vs orElseGet(lazy); never bare get()
parallelStream only for big CPU-bound stateless work
```

Module 3 — Mock Interview (answer, then continue)

1. "Given `List<Order>`, produce total revenue per customer, sorted descending, as a `LinkedHashMap`."
2. "Explain, step by step, what elements flow through `stream().filter(...).map(...).findFirst()` and why `map` may run fewer times than the list size."
3. "`orElse` vs `orElseGet` — when does it matter in production?"
4. "Rewrite a nested loop that flattens and dedups a list of lists using streams."
5. "When would you refuse to use a parallel stream?"

Continue to Module 4 when ready.